Deleted Project Restoration

You should not perform this action

If you need to restore data lost due to GitLab's actions, immediately open an S1 incident using the `/incident` command in Slack.

As a general policy, we do not perform these restores in any other case. You should refuse to perform this restore unless the request is escalated to an Infrastructure manager.

Plan on 4-6 days of effort over a 5-10 day time period for a successful restore. And even then, there is no guarantee of full recovery.

Introduction

As long as we have the database and Gitaly backups, we can restore deleted GitLab projects.

It is strongly suggested to read through this entire document before proceeding.

There is an alternate runbook for a different method of restoring namespaces, but the one in this document should be preferred.

Background

There are two sources of data that we will be restoring: the project metadata (issues, merge requests, members, etc.), which is stored in the main database (Postgres), and the repositories (main and wiki) which are stored on a Gitaly shard.

Container images and CI artifacts are not restored by this process.

Part 1: Restore the project metadata

If a project is deleted in GitLab, it is entirely removed from the database. That is, we also lack the necessary metadata to recover data from file servers. Recovering meta- and project data is a multi-step process:

1. Restore a full database backup and perform point-in-time recovery (PITR)
2. Extract metadata necessary to recover from git/wiki data from file servers
3. Export the project from the database backup and import into GitLab

Special procedure if the deletion was less than 8h ago We run a delayed archive replica of our production database with `recovery_min_apply_delay = '8h'`. It is therefore at least 8 hours behind production. If the deletion request comes quickly enough, we can use the delayed replica to perform a point-in-time recovery (PITR) without spinning up a new restore instance.

This procedure has been validated in INC-8739 (2026-03-25).

There are three delayed replica clusters, all of which must be recovered to the same PIT: Main, CI, and Sec. Because all three databases must be at a consistent point in time for a coherent restore, you almost always need the Full PITR procedure below rather than the simple pause.

Which procedure to use?

Scenario	Procedure
Deletion just happened, replica hasn't replayed it yet, single cluster only, no specific target time needed	Simple pause procedure
Need all three clusters (Main, CI, Sec) at the same specific point in time	Full PITR procedure — almost always required

---

Simple pause procedure (single cluster, no target time)

This procedure is rarely suitable for production restores. As noted above, all three delayed replica clusters must be recovered to the same point in time for a coherent restore. This simple pause procedure should only be used if you need to restore a specific table or set of tables that you know beforehand are independent without dependencies on other databases. Otherwise, use the Full PITR procedure below.

This procedure is documented here for historical reference — it was the original runbook procedure when GitLab had only a single database and may still be useful in edge cases.

Use only if the replica has not yet replayed the deletion and you do not need a specific target time across all clusters.

1. SSH to the delayed replica.
2. Disable `chef-client` to prevent config changes:

   ```
   sudo chef-client-disable "Incident #<INC_NUMBER> - pausing delayed replica"
   ```

   Verify:

   ```
   chef-client-is-enabled   # expect: "chef-client is currently disabled"
   ```

3. Pause WAL replay immediately:

```
SELECT pg_wal_replay_pause();
```

4. Verify replay is paused and lag is increasing:

```
SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;
```

5. Perform the data export.

6. Resume replay:

```
SELECT pg_wal_replay_resume();
```

7. Re-enable `chef-client`:

```
chef-client-enable
```

> `pg_wal_replay_resume()` is **only safe here because no `recovery_target_time` was set**. If you have set a `recovery_target_time` and PostgreSQL has reached it, calling `pg_wal_replay_resume()` will **promote the replica to primary** — use the Full PITR procedure below instead.

Full PITR procedure (Main, CI, and Sec to the same target time)
Use this when you need a consistent recovery point across all three database clusters. This is the standard procedure for group/project restores.

> **Critical — Do NOT call `pg_wal_replay_resume()` once a `recovery_target_time` has been set and reached.** It will promote the replica to primary, making it unable to apply further WAL. This was learned during INC-8739 where the Sec cluster was accidentally promoted and had to be rebuilt. Use `pg_ctl restart` to roll forward instead.

> **Patroni will overwrite `postgresql.auto.conf` on every restart.** Stop Patroni first before setting recovery parameters via `ALTER SYSTEM`.

Pre-flight checks

1. Confirm the deletion timestamp and agree on a recovery target time — it must be **before** the earliest deletion event. Verify using logs (e.g. Kibana/OpenSearch).

2. Create an alert silence for all three delayed replica clusters — replication lag alerts will fire during this procedure.

3. Check the current replay position on each cluster:

```
SELECT
    now()                                   AS current_time,
    pg_last_xact_replay_timestamp()         AS latest_replay_time,
```

```
        now() - pg_last_xact_replay_timestamp()  AS replay_lag,
        pg_is_wal_replay_paused()                AS is_paused;
```

Perform the following steps on **each** of the three clusters (Main, CI, Sec):

Step 1 — Disable chef-client

```
sudo chef-client-disable "Incident #<INC_NUMBER> - performing PITR on delayed replica"
```

Verify:

```
chef-client-is-enabled   # expect: "chef-client is currently disabled"
```

Step 2 — Stop Patroni

```
# Pause Patroni scheduling
gitlab-patronictl pause <cluster-name>

# Stop the Patroni service
sudo systemctl stop patroni.service

# Confirm stopped
sudo systemctl status patroni.service   # expect: inactive (dead)
```

Step 3 — Set recovery target parameters via ALTER SYSTEM

Do this while PostgreSQL is **still running** (before any restart):

```
-- Example from INC-8739 where target was 15:40 UTC on 2026-03-25:
ALTER SYSTEM SET recovery_target_time = '2026-03-25 15:40:00+00';
ALTER SYSTEM SET recovery_target_action = 'pause';
ALTER SYSTEM SET recovery_target_inclusive = 'on';
```

Confirm parameters are written to `postgresql.auto.conf`:

```
cat /var/opt/gitlab/postgresql/data17/postgresql.auto.conf
```

Expected:

```
recovery_target_time = '2026-03-25 15:40:00+00'
recovery_target_action = 'pause'
recovery_target_inclusive = 'on'
```

Step 4 — Restart PostgreSQL via pg_ctl

```
/usr/lib/postgresql/17/bin/pg_ctl -D /var/opt/gitlab/postgresql/data17 restart -m fast --wai
```

> Note: On large instances `pg_ctl` may report `server did not start in time`. Wait a few minutes and check — PostgreSQL typically comes up successfully regardless. Monitor `/var/log/gitlab/postgresql/` logs.

Step 5 — Confirm parameters applied

```
gitlab-psql -c 'SHOW recovery_target_time; SHOW recovery_target_action; SHOW recovery_target
```

Expected:

```
  recovery_target_time
------------------------
 2026-03-25 15:40:00+00

 recovery_target_action
------------------------
 pause

 recovery_target_inclusive
---------------------------
 on
```

Step 6 — Monitor until paused at target time

```
SELECT
    now()                                    AS current_time,
    pg_last_xact_replay_timestamp()          AS latest_replay_time,
    now() - pg_last_xact_replay_timestamp()  AS replay_lag,
    pg_is_wal_replay_paused()                AS is_paused;
```

Keep polling until `is_paused = t`. You will also see this in the PostgreSQL logs confirming the pause:

```
LOG:  recovery stopping before commit of transaction XXXXXXXX, time 2026-03-25 15:40:xx+00
LOG:  pausing at the end of recovery
HINT:  Execute pg_wal_replay_resume() to promote.
```

> The hint about `pg_wal_replay_resume()` is a standard PostgreSQL message in this context — **ignore it and do not run it.**

Step 7 — Confirm paused at correct time

```
SELECT
    pg_is_wal_replay_paused()        AS is_paused,   -- expect: true
    pg_last_xact_replay_timestamp()  AS paused_at;   -- expect: ~your target time
```

Repeat steps 1–7 for all three clusters before proceeding to data export.

Rolling forward (if you need to advance the recovery target) If you need to move the pause point forward by a few minutes after pausing, without losing the replica:

1. While PostgreSQL is still running (paused), update the target:

   ```
   -- Example: advance from 15:40 to 15:43
   ALTER SYSTEM SET recovery_target_time = '2026-03-25 15:43:00+00';
   ```

```
ALTER SYSTEM SET recovery_target_action = 'pause';
ALTER SYSTEM SET recovery_target_inclusive = 'on';
```

2. Restart PostgreSQL:

```
/usr/lib/postgresql/17/bin/pg_ctl -D /var/opt/gitlab/postgresql/data17 restart -m fast
```

3. Monitor and confirm paused at the new time (repeat Step 6–7 above).

`ALTER SYSTEM` writes to `postgresql.auto.conf` which takes precedence over `postgresql.conf` — so this works correctly even if the original parameters were set directly in `postgresql.conf` by a previous operator.

After data export — restoring delayed replicas to normal state Once the data export is complete, restore each cluster back to normal delayed WAL apply. Perform on each cluster:

1. While PostgreSQL is still running, reset the recovery target parameters:

```
ALTER SYSTEM RESET recovery_target_time;
ALTER SYSTEM RESET recovery_target_action;
ALTER SYSTEM RESET recovery_target_inclusive;
```

2. Restart PostgreSQL:

```
/usr/lib/postgresql/17/bin/pg_ctl -D /var/opt/gitlab/postgresql/data17 restart -m fast
```

3. Confirm the replica is back in recovery and replaying normally:

```
SELECT
    pg_is_in_recovery()              AS is_replica,    -- expect: true
    pg_is_wal_replay_paused()        AS is_paused,     -- expect: false
    pg_last_xact_replay_timestamp()  AS resumed_at;    -- should start advancing
```

4. Start Patroni:

```
sudo systemctl start patroni.service
```

5. Confirm cluster is healthy and in archive recovery:

```
gitlab-patronictl list   # expect: Standby Leader | in archive recovery
```

6. Re-enable `chef-client`:

```
chef-client-enable
```

7. Remove the alert silence.

Alternative: PITR via disk snapshot As an alternative to the live delayed replica procedure above, PITR can also be achieved by identifying the correct VM disk snapshot taken before the deletion event and recreating the delayed replica cluster from it via Terraform. This approach was used for the Sec cluster during INC-8739.

1. Identify the disk snapshot taken **before** the deletion event — use the GCP Console or `gcloud` to list snapshots by timestamp and find the correct one:

   ```
   gcloud compute snapshots list --project=gitlab-production \
     --sort-by=~creationTimestamp \
     --format="table(name,creationTimestamp,diskSizeGb)"
   ```

2. Add a temporary Terraform data source pointing to the identified snapshot in the relevant cluster `.tf` file:

   ```
   data "google_compute_snapshot" "gcp_database_snapshot_gprd_<cluster>_pitr" {
     name = "<snapshot-name>"   # e.g. "q9qaqmu7x18s"
   }
   ```

3. Update the cluster resource to use that snapshot temporarily:

   ```
   data_disk_snapshot = data.google_compute_snapshot.gcp_database_snapshot_gprd_<cluster>_
   ```

   > Note: By default the cluster uses a dynamic snapshot reference such as `data.google_compute_snapshot.gcp_database_snapshot_gprd_main.id`. This is a **temporary override** for the duration of the incident only — revert it once the restore is complete.

4. Apply Terraform to recreate the VM from that snapshot.

5. Set the recovery target parameters (`recovery_target_time`, `recovery_target_action = 'pause'`, `recovery_target_inclusive = 'on'`) via `ALTER SYSTEM` or directly in `postgresql.conf`, then start PostgreSQL and monitor until it pauses at the target time.

6. Proceed with the data export, then revert the Terraform change to restore normal operation.

This approach can be faster than waiting for the live delayed replica to replay WAL to the target time, particularly if the target time is many hours ago. It also avoids any risk to the live delayed replica cluster.

> See also: Restore full database backup and perform PITR for the full pipeline-based restore approach using WAL-G.

Restore full database backup and perform PITR If the request arrived promptly and you were able to follow the special procedure above, skip this section.

In order to restore from a database backup, we leverage the backup restore pipeline in “gitlab-restore” project. It can be configured to start a new GCE in-

stance and restore a backup to an exact point in time for later recovery (example MR). Currently, Postgres backups are created by WAL-G. Use `WAL_E_OR_WAL_G` CI variable to switch to WAL-E if needed (see below).

1. Push a commit similar to the example MR above. Note that you don't need to create an MR although you can if you like.
2. You can start the process in CI/CD Pipelines of the "gitlab-restore" project.
3. Select your branch, and configure the variables as detailed in the steps below.
4. To perform PITR for the production database, use CI/CD variable `ENVIRONMENT` set to `gprd`. The default value is `gstg` meaning that the staging database will be restored.
5. To ensure that your instance won't get destroyed in the end, set CI/CD variable `NO_CLEANUP` to `1`.
6. In CI/CD Pipelines, when starting a new pipeline, you can choose any Git branch. But if you use something except `master`, there are high chances that production DB copy won't fit the disk. So, use `GCE_DATADISK_SIZE` CI/CD variable to provision an instance with a large enough disk. As of January 2020, we need to use at least `6000` (6000 GiB). Check the `GCE_DATADISK_SIZE` value that is currently used in the backup verification schedules (see CI/CD Schedules).
7. By default, an instance of `n1-standard-16` type will be used. Such instances have "good enough" IO throughput and IOPS quotas Google throttles disk IO based on the disk size and the number of vCPUs). In the case of urgency, to get the best performance possible on GCP, consider using `n1-highcpu-32`, specifying CI/CD variable `GCE_INSTANCE_TYPE` in the CI/CD pipeline launch interface. It is highly recommended to check the current resource consumption (total vCPUs, RAM, disk space, IP addresses, and the number of instances and disks in general) in the GCP quotas interfaces of the "gitlab-restore" project.
8. It is recommended (although not required) to specify the instance name using CI/CD variable `INSTANCE_NAME`. Custom names help distinguish GCE instances from auto-provisioned and from provisioned by someone else. An excellent example of custom name: `nik-gprd-infrastructure-issue-1234` (means: requested by `nik`, for environment `gprd`, for the the `infrastructure` issue `1234`). If the custom name is not set, your instance gets a name like `restore-postgres-gprd-XXXXX`, where `XXXXX` is the CI/CD pipeline ID.
9. As mentioned above, by default, WAL-G will be used to restore from a backup. It is controlled by CI variable `WAL_E_OR_WAL_G` (default value: `wal-g`). If WAL-E is needed, set CI variable `WAL_E_OR_WAL_G` to `wal-e`, but expect that restoring will take much more time. For the "restore basebackup" phase, on `n1-standard-16`, the expected speed of filling the PGDATA directory is 0.5 TiB per hour for WAL-E and 2 TiB per hour for WAL-G.

10. The default WAL source path, `BACKUP_PATH`, will most likely be different from what is defined as a default variable for pipelines. You will need to examine a production Patroni Main VM to determine the WAL bucket. It's most likely found in `/etc/wal-g.d/env/WALG_GS_PREFIX`. The `BACKUP_PATH` is just the section after the `gs://gitlab-gprd-postgres-backup/` part of the storage path.
11. To control the process, SSH to the instance. The main items to check:
    - `df -hT /var/opt/gitlab` to ensure that the disk is not full (if it hits 100%, it won't be noticeable in the CI/CD interfaces, unfortunately),
    - `sudo journalctl -f` to see basebackup fetching and, later, WAL fetching/replaying happening.
12. Finally, especially if you have made multiple attempts to provision an instance via CI/CD Pipelines interface, check VM Instances in GCP console to ensure that there are no stalled instances related to your work. If there are some, delete them manually. If you suspect that your attempts failing because of some WAL-G issues, try WAL-E (see above).

The instance will progress through a series of operations:

1. The basebackup will be downloaded
2. The Postgres server process will be started, and will begin progressing past the basebackup by recovering from WAL segments downloaded from GCS.
3. Initially, Postgres will be in crash recovery mode and will not accept connections.
4. At some point, Postgres will accept connections, and you can check its recovery point by running `select pg_last_xact_replay_timestamp();` in a `gitlab-psql` shell.
5. Check back every 30 minutes or so until the recovery point you wanted has been reached. You don't need to do anything to stop further recovery, your branch has configured it to pause at this point.

After the process completes, an instance with a full GitLab installation and a production copy of the database is available for the next steps.

Note that the startup script will never actually exit due to the branch configuration that causes Postgres to pause recovery when some point is reached. It loops forever waiting for a recovery point equal to script start time.

Export project from database backup and import into GitLab Here, we use the restored database instance with a GitLab install to export the project through the standard import/export mechanism. We want to avoid starting a full GitLab instance (to perform the export throughout the UI) because this sits on a full-sized production database. Instead, we use a rails console to trigger the export.

1. Start Redis: `gitlab-ctl start redis` (Redis is not going to be used really, but it's a required dependency)

2. Start Rails: `gitlab-rails console`

Modify the literals in the following console example and run it. This retrieves a project by its ID, which we obtain by searching for it by namespace ID and its name. We also retrieve an admin user. Use yours for auditability. The ProjectTreeSaver needs to run "as a user", so we use an admin user to ensure`that we have permissions. Some additional data that is not part of the export will be saved to a YAML file in`/tmp/project–additional-data.yml` to be used after importing.

```
user = User.find_by_username('an-admin')
project = Project.find_by_full_path('namespace/project-name')
additional_data = {}

project.repository_storage
# Note down this output.

project.disk_path
# Note down this output.

additional_data[:created_at] = project.created_at
additional_data[:issue_authors] = project.issues.pluck(:iid, :author_id).to_h
additional_data[:mr_authors] = project.merge_requests.pluck(:iid, :author_id).to_h
additional_data[:pipeline_users] = project.ci_pipelines.pluck(:iid, :user_id).to_h
additional_data[:ci_variables] = project.variables.select(:key, :value, :protected, :masked
additional_data[:deploy_tokens] = project.deploy_tokens.map { _1.attributes.except('id', 'pr
additional_data[:forks] = project.forks.map(&:id)

File.write("/tmp/project-#{project.id}-additional-data.yml", additional_data.to_yaml)

shared = Gitlab::ImportExport::Shared.new(project)
version_saver = Gitlab::ImportExport::VersionSaver.new(shared: shared)
tree_saver = Gitlab::ImportExport::Project::TreeSaver.new(project: project, current_user: us
version_saver.save
tree_saver.save

include Gitlab::ImportExport::CommandLineUtil
archive_file = File.join(shared.archive_path, Gitlab::ImportExport.export_filename(exportabl
tar_czf(archive: archive_file, dir: shared.export_path)
# Some output that includes the path to a project tree directory, which will be something li
```

We now have the Gitaly shard and path on persistent disk the project was stored on, and if the final command succeeded, we have a project metadata export JSON.

It's possible that the save command failed with a "storage not found" error. If this is the case, edit `/etc/gitlab/gitlab.rb` and add a dummy entry to `git_data_dirs` for the shard, then run `gitlab-ctl reconfigure`, and restart

the console session. We are only interested in the project metadata for now, but the `Project#repository_storage` must exist in config.

An example of the `git_data_dirs` config entry in `gitlab.rb`:

```
git_data_dirs({
  "default" => {
    "path" => "/mnt/nfs-01/git-data"
   },
  "nfs-file40" => {
    "path" => "/mnt/nfs-01/git-data"
   }
})
```

You can safely duplicate the path from the `default` git_data_dir, it doesn't matter that it won't contain the repository.

Make `project.json` abd `project-<id>-additional-data.yml` accessible to your `gcloud ssh` user:

```
mv /path/to/project/tree /tmp/
chmod -R 644 /tmp/project /tmp/project-<id>-additional-data.yml
```

Download both files, replacing the project and instance name in this example as appropriate:

```
gcloud --project gitlab-restore compute scp --recurse restore-postgres-gprd-88895:/tmp/proje
gcloud --project gitlab-restore compute scp restore-postgres-gprd-88895:/tmp/project-<id>-ad
```

Download a stub exported project.

On your local machine, replace the project.json inside the stub archive with the real one:

```
mkdir repack
tar -xf blank_export.tar.gz -C repack
cd repack
cp ../project.json ./
tar -czf ../repacked.tar.gz ./
```

Log into gitlab.com using your admin account, and navigate to the namespace in which we are restoring the project. Create a new project, using the "from GitLab export" option. Name it after the deleted project, and upload repacked.tar.gz.

A project can also be imported on the command line.

This will create a new project with equal metadata to the deleted one. It will have a stub repo and wiki. The persistent object itself is new: it has a new project_id, and the repos are not necessarily stored on the same Gitaly shard, and will have new disk_paths.

Browse the restored project's member list. If your admin account is listed as a maintainer, leave the project.

Upload `project-<id>-additional-data.yml` to the production console node:

```
scp project-<id>-additional-data.yml console-01-sv-gprd.c.gitlab-production.internal:/tmp/
```

Start a Rails console, run the following script to update the project's data that wasn't included in the export:

```
project = Project.find_by_full_path('namespace/project-name')
additional_data = YAML.unsafe_load(File.read("/tmp/project-#{project.id}-additional-data.yml

# Restore creation timestamp
project.created_at = additional_data[:created_at]
project.save!

# Restore issue authors
project.issues.in_batches(of: 100) do |issues|
    issues.each do |issue|
        next unless additional_data[:issue_authors][issue.iid]
        issue.author_id = additional_data[:issue_authors][issue.iid]
        issue.save
    end
end

# Restore merge requests authors
project.merge_requests.in_batches(of: 100) do |merge_requests|
    merge_requests.each do |merge_request|
        next unless additional_data[:mr_authors][merge_request.iid]
        merge_request.author_id = additional_data[:mr_authors][merge_request.iid]
        merge_request.save
    end
end

# Restore CI pipeline creators
project.ci_pipelines.in_batches(of: 100) do |pipelines|
    pipelines.each do |pipeline|
        next unless additional_data[:pipeline_users][pipeline.iid]
        pipeline.user_id = additional_data[:pipeline_users][pipeline.iid]
        pipeline.save!
    end
end

# Restore CI variables
additional_data[:ci_variables].each do |ci_var|
    project.variables.create!(ci_var)
end

# Restore deploy tokens
```

```ruby
additional_data[:deploy_tokens].each do |token|
    project.deploy_tokens.create!(token.merge(project_id: project.id))
end

# Restore fork relations
Project.where(id: additional_data[:forks]).in_batches(of: 100) do |projects|
    projects.each do |forked_project|
        forked_project.forked_from_project = project
        forked_project.save
    end
end
Projects::ForksCountService.new(project).refresh_cache

# Note down the following two values. These point us to the stub repo (and wiki repo) that u
project.repository_storage
project.disk_path
```

Part 2: Restore Git repositories

The first step is to check if the repositories still exist at the old location. They likely do not, but it's possible that unlike project metadata they have not (yet) been removed.

Using the repository_storage and disk_path obtained from the DB **backup** (i.e. for the old, deleted project metadata), ssh into the relevant Gitaly shard and navigate to `/var/opt/gitlab/git-data/repositories`. Check if `<disk_path>.git` and `<disk_path>.wiki.git` exist. If so, create a snapshot of this disk in the GCE console.

If these directories do not exist, browse GCE snapshots for the last known good snapshot of the Gitaly persistent disk on which the repository used to be stored.

Either way, you now have a snapshot with which to follow the next steps.

Restoring from Disk Snapshot Run all commands on the server in a root shell.

1. Create a new disk from the snapshot. Give it a relevant name and description, and ensure it's placed in the same zone as the Gitaly shard referenced by the **new** project metadata (i.e. that obtained from the production console).
2. In the GCE console, edit the Gitaly shard on which the new, stub repositories are stored. Attach the disk you just created with a custom name "pitr".
3. GCP snapshots are not guaranteed to be consistent. Check the filesystem: `fsck.ext4 /dev/disk/by-id/google-pitr`. If this fails, do not necessarily stop if you are later able to mount it: the user is already missing their repository, and if we are lucky the part of the filesystem containing

it is not corrupted. Later, we ask the customer to check the repository, including running `git fsck`. Unfortunately it's possible that the repository would already have failed this check, and we can't know.

4. Mount the disk: `mkdir /mnt/pitr; mount -o ro /dev/disk/by-id/google-pitr /mnt/pitr`
5. Navigate to the parent of `/var/opt/gitlab/git-data/repositories/<disk_path>.git`.
6. `mv <new_hash>.git{,moved-by-your-name}`, and similar for the wiki repository. Reload the project page and you should see an error. This double-checks that you have moved the correct repositories (the stubs). You can `rm -rf` these directories now. `<new_hash>` refers to the final component of the new `disk_path`.
7. `cp -a /mnt/pitr/git-data/repositories/<old_disk_path>.git ./<new_hash>.git`, and similarly for the wiki repo. If you've followed the steps up to now your CWD is something like `/var/opt/gitlab/git-data/repositories/@hashed/ab/cd`
8. Reload the project page. You should see the restored repository, and wiki.
9. `umount /mnt/pitr`
10. In the GCE console, edit the Gitaly instance, removing the pitr disk.

Part 3: Check in with the customer

Once the customer confirms everything is restored as expected, you can delete any disks, and Postgres PITR instances created by this process.

It might be worth asking the customer to check their repository with `git fsck`. If the filesystem-level fsck we ran on the Gitaly shard succeeded, then the result of `git fsck` doesn't matter that much: the repository might already have been corrupted, and that's not necessarily our fault. However, if both `fsck`s failed, we can't know whether the corruption predated the snapshot or not.

Troubleshooting

Rails console errors out with a stacktrace

You may need to copy the production `db_key_base` key into the restore node. You can find the key in `/etc/gitlab/gitlab-secrets.json`.

Once you've added this key, you will need to run `gitlab-ctl reconfigure` to get the changes working and a working console.

Gitlab reconfigure fails

You may need to edit the `/etc/gitlab/gitlab.rb` file and disable object_store like this: `gitlab_rails['object_store']['enabled'] = false`

Copying the Git repository results in a bad fsck or non-working repository

You may be recovering a repository with objects in a shared pool. Try to re-copy using these instructions.